

Recovery Score - Technical Reference

System architecture, data flow, formulas, and exact weightage used in production

For engineering / app-dev use. Companion to the plain-language "Recovery Score Explained" doc.

1. File Map

Every file involved in the Recovery feature, and what it owns:

File	Responsibility
server/services/recoveryScoreService.js	Core scoring engine - all math lives here. Entry point: calculateRecoveryScore(userId, dateStr).
server/services/recoveryBaselineService.js	Shared helper - pulls N days of a metric, merges multi-device same-day readings, computes mean/SD.
server/services/recoveryRecommendationService.js	Rule-based (not AI) lookup: band + activity load -> recommendation text + suggested activities.
server/services/recoveryPatternService.js	Scans last 30 days for 2 pre-defined correlations (frequency count, not ML).
server/services/recoveryAnalyticsService.js	Read-side: serves daily/weekly/monthly/yearly views, personal baseline range, patterns.
server/config/activityCatalog.js	Single source of truth for activity types + MET values; also powers recovery-friendly activity filtering.
server/models/RecoveryDailySummary.js	One document per user per day - the persisted output of calculateRecoveryScore().
server/models/StressDailySummary.js	Source of avgHrvMs (real HRV, ms) - rolled up per day from raw StressSample.
server/models/HeartRateDailySummary.js	Source of restingBpm.value (RHR) per day.
server/models/VitalsDailySummary.js	Source of avgRespiratoryRate (RR) per day.
server/models/SleepSession.js	Source of totalSleepMinutes, awakeMinutes, bedTime per night.
server/models/DailyActivityMetric.js	Source of activeMinutes per day (drives the Activity modifier).
server/controllers/wearableController.js	getRecoveryAnalyticsData - the actual HTTP endpoint the app calls.

2. End-to-End Data Flow

```
Apple Watch / HealthKit
|
v
POST /wearables/sync-os-health (wearableController.syncOsHealthData)
|
v
wearableIngestService.js
  applyStressSamples() -> StressDailySummary.avgHrvMs (running avg, per day)
  applyHeartRateSamples() -> HeartRateDailySummary.restingBpm
  applyVitalsSamples() -> VitalsDailySummary.avgRespiratoryRate
  applySleepSessions() -> SleepSession (totalSleepMinutes, awakeMinutes, bedTime)
  (DailyActivityMetric updated by the existing activity-sync path)
  |
  v
Triggered on write: scoreRecompute.triggerRecoveryScoreRecompute(userId, date)
|
```

```

      v
recoveryScoreService.calculateRecoveryScore(userId, dateStr)
  1. buildHrvEngine() -> today's HRV vs 14-day & 90-day baseline
  2. buildRhrEngine() -> today's RHR vs 14-day & 90-day baseline
  3. buildRrEngine() -> today's RR vs 14-day baseline
  4. buildSleepEngine() -> Duration + Efficiency + Regularity
  5. buildActivityModifier() -> 7-day vs 28-day load ratio
  6. Combine -> Physiology -> RecoveryBase -> +Activity modifier
  7. computeBaselineDrift() -> flags depressed long-term baseline
  8. Apply Recovery Ceiling if drifted
  9. buildSafetyWarnings() -> independent absolute-threshold check
  10. buildRecommendation() -> rule-based text + suggested activities
      |
      v
RecoveryDailySummary.findOneAndUpdate() (upsert, one doc per user per day)
      |
      v
GET /wearables/recovery/analytics?range=... (recoveryAnalyticsService.getRecoveryAnalytics)
      |
      v
Mobile app

```

3. Baseline Calculation - The Core Primitive

[server/services/recoveryBaselineService.js](#)

Every engine (HRV, RHR, RR) calls the same function to get a baseline:

```

getWindowStats(Model, userId, valuePath, dateStr, windowDays)

1. Query window: [dateStr - windowDays, dateStr) <- TODAY IS EXCLUDED
2. If a user has multiple devices reporting the same day, average them
   (point measurements like bpm/ms are averaged, never summed)
3. Compute sample mean and sample standard deviation (n-1 denominator):

mean = sum(values) / n
sd = sqrt( sum((v - mean)^2) / (n - 1) ) [sd = null if n < 2]

Returns: { mean, sd, n }

```

Two windows, used together

Window	Constant	Used for
Recent	RECENT_WINDOW_DAYS = 14	"How does today compare to my last two weeks" - matches Oura's public 14-day convention.
Long-term	LONG_WINDOW_DAYS = 90	"How does today compare to my deeper, more stable normal" - protects against a temporarily bad recent window looking like full recovery.

Z-score -> T-score conversion

```
z = (today_value - baseline_mean) / baseline_sd
```

```
T = clamp( 50 + 10*z , 0, 100 ) // standard psychometric T-score:  
                                // mean=50, SD=10. NOT an invented scale.  
                                // Same transform used in clinical  
                                // instruments (e.g. MMPI).
```

NOTE: z requires sd > 0 (at least 2 valid baseline days). With fewer, the function returns null and that sub-score is excluded from the weighted average below (see Section 6).

4. Per-Metric Engine Logic

4.1 HRV Engine - buildHrvEngine()

Source: StressDailySummary.avgHrvMs. Direction: higher HRV vs. baseline = better (no sign flip).

```
recentZ = zScore(todayHrv, baseline14.mean, baseline14.sd)
longZ = zScore(todayHrv, baseline90.mean, baseline90.sd)

recentScore = TScore(recentZ)
longScore = TScore(longZ)

HRVScore = 0.40 * recentScore + 0.60 * longScore
            (renormalized if either side is null - see Section 6)
```

NOTE: 60% weight on the long-term window is deliberate: a single depressed recent window should not be able to quickly redefine what counts as "healthy."

4.2 RHR Engine - buildRhrEngine()

Source: HeartRateDailySummary.restingBpm.value. Direction: LOWER RHR vs. baseline = better (sign flipped).

```
recentZ = zScore(todayRhr, baseline14.mean, baseline14.sd)
longZ = zScore(todayRhr, baseline90.mean, baseline90.sd)

recentScore = TScore( -recentZ ) // note the sign flip
longScore = TScore( -longZ )

RHRScore = 0.40 * recentScore + 0.60 * longScore
```

4.3 Respiratory Rate Engine - buildRrEngine()

Source: VitalsDailySummary.avgRespiratoryRate. Direction: deviation in EITHER direction is bad (only a 14-day window is used - RR is treated as a stability check, not a trend metric).

```
z = zScore(todayRr, baseline14.mean, baseline14.sd)

RRScore = TScore( -abs(z) ) // deviation either way lowers the score
```

4.4 Sleep Engine - buildSleepEngine()

Source: SleepSession (totalSleepMinutes, awakeMinutes, bedTime). Three real sub-components:

Sub-component	Formula	Weight
Duration	100 - abs(hoursSlept - 8) * 15, clamped 0-100	0.45
Efficiency	totalSleepMinutes / (totalSleepMinutes + awakeMinutes) * 100	0.30
Regularity	100 - (SD of bedtime-minutes over last 14 nights / 120) * 100	0.15
Continuity (NOT built)	Would need a per-night wake-event COUNT - HealthKit sleep stages only give total awakeMinutes, not how many separate times the user woke.	0.10 (unused - renormalized away)

```
SleepScore = weightedAverage([
  { score: durationScore, weight: 0.45 },
  { score: efficiencyScore, weight: 0.30 },
  { score: regularityScore, weight: 0.15 },
]) // renormalized over whichever of the 3 are available
```

4.5 Activity Modifier - buildActivityModifier()

Source: DailyActivityMetric.activeMinutes. This is a MODIFIER (added/subtracted at the end), not a weighted component - it never earns bonus points, only applies a penalty for overload.

```
acuteLoad = avg(activeMinutes, last 7 days)
chronicLoad = avg(activeMinutes, last 28 days)
loadRatio = acuteLoad / chronicLoad

if loadRatio > 1.2:
  modifier = -min(12, round((loadRatio - 1.2) * 20)) // capped at -12
else:
  modifier = 0 // underactivity never gets a bonus, only ever neutral
```

5. Weightage - Full Hierarchy

Three levels of combination, each independently renormalized if a piece is missing (Section 6).

```
LEVEL 1 - Physiology
  Physiology = 0.50 * HRVScore + 0.40 * RHRScore + 0.10 * RRScore

LEVEL 2 - Recovery Base
  RecoveryBase = 0.70 * Physiology + 0.30 * SleepScore

LEVEL 3 - Final Score
  RecoveryRaw = RecoveryBase + ActivityModifier
  RecoveryScore = clamp(round(RecoveryRaw), 0, 100)
  RecoveryScore = min(RecoveryScore, RECOVERY_CEILING) IF baseline is drifted (Section 8)
```

Constant	Value	File	Status
HRV weight (within Physiology)	0.50	recoveryScoreService.js	Placeholder - directionally matches Whoop's public HRV-dominant disclosure
RHR weight (within Physiology)	0.40	recoveryScoreService.js	Placeholder
RR weight (within Physiology)	0.10	recoveryScoreService.js	Placeholder - matches Polar's public "breathing rate smallest weight" disclosure
Physiology weight (within RecoveryBase)	0.70	recoveryScoreService.js	Placeholder
Sleep weight (within RecoveryBase)	0.30	recoveryScoreService.js	Placeholder
HRV recent/long split	0.40 / 0.60	buildHrvEngine()	Placeholder
RHR recent/long split	0.40 / 0.60	buildRhrEngine()	Placeholder
Sleep sub-weights	0.45 / 0.30 / 0.15	buildSleepEngine()	Placeholder

NOTE: Every row above is labeled "Placeholder" on purpose - no wearable brand publishes a validated weighting (confirmed via a 2025 peer-reviewed comparison study). These are our starting-point judgment calls, directionally grounded in what brands DO disclose, not empirically fitted to our own users yet. Updating them later means changing these constants only - the surrounding logic (z-score, renormalization, ceiling) does not need to change.

6. Missing-Data Handling - weightedAverage()

Every weighted combination in this system (Physiology, RecoveryBase, Sleep) goes through the same helper, which renormalizes over only the parts that are actually available:

```
function weightedAverage(parts):
  available = parts.filter(p => p.score != null)
  if available.length == 0: return null
  totalWeight = sum(p.weight for p in available)
  return sum(p.score * (p.weight / totalWeight) for p in available)
```

```
Example: HRV missing, only RHR (0.40) and RR (0.10) available -> renormalized to:  
RHR effective weight = 0.40 / 0.50 = 0.80  
RR effective weight = 0.10 / 0.50 = 0.20
```

NOTE: Renormalization happens WITHIN each domain independently (Physiology, then RecoveryBase) - a metric missing inside Physiology never borrows weight from Sleep or vice versa.

The "Day 1" edge case (fixed bug)

A reading existing today is not the same as having a baseline to score it against - with zero baseline history, z-score/T-score both return null, which can cascade to Physiology = null and RecoveryBase = null. Left unguarded, RecoveryRaw = null + activityModifier coerces to 0 in JavaScript, showing a brand-new user "Recovery: 0/100" on their very first day.

```
if (recoveryBase == null):  
    save({ recoveryScore: null, ... })  
    return { recoveryScore: null, status: 'insufficient_baseline', confidence, ... }
```

7. Score Bands - classifyRecoveryBand()

```
if score >= 85: 'optimal' "Optimal Recovery"
if score >= 70: 'moderate' "Moderate Recovery"
if score >= 50: 'low' "Low Recovery"
else: 'very_low' "Very Low Recovery"
```

Thresholds (85 / 70 / 50) match Oura's published Readiness tiers exactly, extended to 4 tiers for the product design's "Moderate Recovery" copy.

8. Safety Floor, Baseline Drift, and Recovery Ceiling

8.1 Safety Floor - buildSafetyWarnings() (independent of the score)

```
SAFETY_FLOOR = {
  hrvMsLow: 20, // below this is low for any healthy adult
  rhrBpmHigh: 100, // resting tachycardia range
  rhrBpmLow: 40, // bradycardia range
  spo2Low: 92, // WHO / pulse-oximetry hypoxia reference
}
// These fire regardless of personal baseline - a score can look "fine"
// relative to a chronically-unhealthy personal history while still
// tripping an absolute warning.
```

8.2 Baseline Drift - computeBaselineDrift()

```
hrvDrifted = hrv.longTermZ <= -DRIFT_Z_THRESHOLD // today still far below 90-day mean
rhrDrifted = rhr.longTermZ >= DRIFT_Z_THRESHOLD // today still far above 90-day mean

baselineStatus = (hrvDrifted or rhrDrifted) ? 'depressed_recent_baseline' : 'stable'

DRIFT_Z_THRESHOLD = 1.5 // PLACEHOLDER - not calibrated
```

8.3 Recovery Ceiling

```
if baselineStatus == 'depressed_recent_baseline' and recoveryScore >
RECOVERY_CEILING_WHEN_DRIFTED:
  recoveryScore = RECOVERY_CEILING_WHEN_DRIFTED
  ceilingApplied = true

RECOVERY_CEILING_WHEN_DRIFTED = 69 // PLACEHOLDER - sits just under
                                     // the Moderate band so a capped
                                     // day reads as "Low," not merely
                                     // "less than Optimal."
```

NOTE: The Ceiling only ever caps DOWN, never raises a score, and only engages when the long-term z-scores (already computed for Physiology, no extra query) indicate the drift condition.

9. Recommendation, Pattern Detection, and Activity Filtering

9.1 Recommendation Engine (rule-based, NOT AI)

[server/services/recoveryRecommendationService.js](#)

A fixed lookup table keyed on band.key, with one dynamic override:

```
BAND_COPY = {
  optimal: { headline: "Great day to push", movement: "30-45 min, moderate-high" },
  moderate: { headline: "Moderate day", movement: "20-30 min, light-moderate" },
  low: { headline: "Take it easier", movement: "10-20 min, light" },
  very_low: { headline: "Prioritize rest", movement: "0-15 min, very light" },
}
if activity.loadRatio > 1.2:
  summary = "Your recent activity has been higher than usual. " + copy.summary
```

9.2 Pattern Detection (frequency count, NOT machine learning)

[server/services/recoveryPatternService.js](#)

Scans the last 30 days for exactly 2 pre-defined correlations. A pattern is only surfaced if it occurred at least MIN_OCCURRENCES=3 times - otherwise it's noise, not a pattern.

Pattern 1: 2 consecutive high-activity days (> 1.3x the user's own 30-day average) followed by a 'low' or 'very_low' band day.

Pattern 2: A night under 390 minutes (6h30m) followed by a 'low' or 'very_low' band day.

9.3 Recovery-Friendly Activity Filter

[server/config/activityCatalog.js](#) -> [getRecoveryFriendlyActivities\(bandKey\)](#)

Uses the published ACSM/CDC MET-intensity classification (light <3.0, moderate 3.0-5.9, vigorous >=6.0 METs) as the ONLY thresholds - not an invented split:

```
metRangeForBand(bandKey):
  optimal -> MET >= 6.0 (vigorous)
  moderate -> 3.0 <= MET < 6.0 (moderate)
  low / very_low -> MET < 3.0 (light)
```

Returns the 4 lowest-MET catalog entries within that range, sorted ascending.

NOTE: Two catalog entries (Stretching/Mobility, Slow Walk - both 2.5 MET, sourced from the 2024 Adult Compendium of Physical Activities) were added specifically because the Light bucket (<3.0 MET) previously only contained Yoga.

10. API Response Shape

GET /wearables/recovery/analysis?range=daily

```
{
  "success": true,
  "range": "daily",
  "entries": [{
    "date": "2026-09-24",
    "recoveryScore": 69,
    "band": { "key": "low", "label": "Low Recovery" },
    "confidence": "good",
    "baselineStatus": "depressed_recent_baseline",
    "warnings": [{ "code": "hrv_below_reference_range", "message": "..."}],
    "components": { "hrv": 59, "restingHeartRate": 61, "respiratoryRate": 47,
      "sleepContribution": 96, "strainContribution": 100 },
    "metricDetails": { "physiology": {...}, "hrv": {...}, "rhr": {...},
      "rr": {...}, "sleep": {...}, "activity": {...},
      "ceilingApplied": true },
    "recommendation": { "headline": "...", "primaryRecommendation": {...},
      "recommendedActivities": [...] }
  ]},
  "personalBaselineRange": { "low": 68, "high": 83 },
  "patterns": [{ "text": "...", "basis": "Observed 4 times in the last 30 days" }]
}
```

NOTE: All Phase 2 fields (band, confidence, warnings, baselineStatus, metricDetails, recommendation, personalBaselineRange, patterns) are strictly additive. A client reading only date/recoveryScore/components sees no behavior change; they'll simply be undefined on rows computed before this shipped.

11. Full Constants Reference

Constant	Value	Location
RECENT_WINDOW_DAYS	14	recoveryScoreService.js
LONG_WINDOW_DAYS	90	recoveryScoreService.js
ACUTE_LOAD_WINDOW_DAYS	7	recoveryScoreService.js
CHRONIC_LOAD_WINDOW_DAYS	28	recoveryScoreService.js
DRIFT_Z_THRESHOLD	1.5	recoveryScoreService.js
RECOVERY_CEILING_WHEN_DRIFTED	69	recoveryScoreService.js
SAFETY_FLOOR.hrvMsLow	20 ms	recoveryScoreService.js
SAFETY_FLOOR.rhrBpmHigh / Low	100 / 40 bpm	recoveryScoreService.js
SAFETY_FLOOR.spo2Low	92%	recoveryScoreService.js
Activity overload threshold	loadRatio > 1.2	buildActivityModifier()

Constant	Value	Location
Activity modifier cap	-12	buildActivityModifier()
Confidence tiers	<7 / 7-13 / 14-27 / 28-59 / 60+ days	confidenceFromBaselineDays()
Pattern MIN_OCCURRENCES	3 (in last 30 days)	recoveryPatternService.js
Recovery-friendly suggestion count	4 activities	activityCatalog.js

Companion document: "Recovery Score Explained" (plain-language, non-technical). This document is the engineering reference - update it whenever a constant or formula changes.